



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

Pthread/OpenMP 编程实验

姓名：吕昊远

学号：2410668

专业：计算机科学与技术

指导教师：王刚

2026 年 5 月 29 日

目录

一、 实验背景	1
二、 实验原理	1
(一) 多线程并行化	1
(二) 线程池	2
(三) IVF (Inverted File Index) 算法	2
(四) IVF-PQ 算法	3
(五) HNSW (Hierarchical Navigable Small World) 算法	3
三、 程序设计与实现	5
(一) Flat-SIMD 多线程编程 (S0~S2)	5
(二) PQ-SIMD 多线程编程 (S3~S6)	7
(三) IVF-SIMD 多线程编程 (S7~S9)	8
(四) IVF-PQ (S10~S15)	10
(五) HNSW 进阶 (S16~S18)	11
(六) 离线索引构建的并行策略	12
四、 实验结果分析	12
(一) 数据总览	12
(二) 线程数分析	14
(三) 参数权衡	16
(四) perf 分析	19
(五) 问题探讨	21
五、 总结与体会	22
六、 附录: 代码仓库链接	23

一、实验背景

在上一次的 SIMD 实验中，我们围绕近似最近邻搜索 (ANNS) 完成了基于 ARM NEON 的向量化优化，通过 Flat-SIMD、SQ-SIMD 以及最终的 PQ-Final 方案，在保持召回率的前提下取得了 15 倍以上的加速比。然而，这些优化均局限于单核的指令级并行，并未充分挖掘多核处理器的潜力。

本次实验在 SIMD 实验的基础上引入多线程并行优化，主要包含以下内容：

- 在已有的 Flat-SIMD 与 PQ-Final 算法基础上(详见 lab2),分别使用 **OpenMP** 和 **Pthread** 实现多线程并行，测量多线程带来的加速比，并分析两种并行框架的特性差异；
- 实现 **IVF (Inverted File Index)** 算法，探索簇级并行策略与动态任务调度的效果；
- 将 IVF 与 PQ 结合，提出 **IVF-PQ** 的两种实现方案（全局 PQ 与簇内独立 PQ），并分析二者在召回率与性能上的差异；
- 作为进阶要求，实现基于 **HNSW** 图索引的多子图点并行搜索，探索图索引在多线程场景下的加速潜力与实际优化效果；

实验使用 DEEP100K 数据集，在实验提供的服务器环境上（ARM 架构）运行与测试，以已经实现了 SIMD 的 S0（Flat-SIMD 单线程）作为本次实验的基准，对上述的实验内容进行实现以及系统性的分析与评估。

二、实验原理

（一）多线程并行化

OpenMP: OpenMP 是一种基于编译器指令 (pragma) 的共享内存并行编程模型。其核心优势在于其内置了线程池，线程仅需创建一次，后续每次进入 `#pragma omp parallel` 区域时直接唤醒，避免了重复创建与销毁的开销。调度策略也由运行时库自动管理，实际使用中简单便捷，代码复杂度较低，适合对规则的数据块进行并行化。

Pthread: POSIX 线程 (Pthread) 提供了更底层的线程控制接口。与 OpenMP 相比，Pthread 需要手动管理线程的创建、同步 (mutex / barrier / condition variable) 和销毁。在本实验中，Pthread 的动态调度通过共享计数器加互斥锁实现，即每个线程在循环中原子地取出下一个任务编号，从而模拟 OpenMP `schedule(dynamic, 1)` 的行为。Pthread 的优势在于其自身的灵活性，在实际编程中，可以根据实际问题来精确控制调度策略与同步粒度等。

两种框架的对比: OpenMP 代码简洁，隐式维护线程池，在计算量较大、任务粒度均匀的场景下优势明显；Pthread 则在任务粒度不均（如 IVF 中各簇大小差异显著）时，通过手动动态调度取得更好的负载均衡效果，在线程数较多时往往能优于 OpenMP。

多线程加速比用 $S_p = T_1/T_p$ 衡量，其中 T_1 为单线程基线延迟， T_p 为 p 线程下的延迟。根据 Amdahl 定律，若程序中串行部分占比为 α ，则理论上的加速上限为：

$$S_p \leq \frac{1}{\alpha + \frac{1-\alpha}{p}}$$

在本次实验中，将对两种多线程技术分别进行实现，通过对比两者的实际实现与优化效果，深入分析不同多线程策略带来的差异与特征。

(二) 线程池

在实际编程过程中，我们可以发现，每次查询都会创建若干线程、执行计算、再销毁线程。对于 2000 次查询、4 个线程的典型场景，这意味着 **8000 次** `pthread_create` / `pthread_join` 调用。而每次的线程创建都需陷入内核态并分配栈空间，开销可达 50~300 μs ，足以完全掩盖并行计算的收益，从而导致 Pthread 版本的多线程编程出现严重负优化。

对于这一问题，我们可以使用线程池来进行解决。在查询循环外一次性创建所有所需的线程，让它们在条件变量（`pthread_cond_t`）上休眠。每次查询只需调用 `pool.dispatch(fn)`，内部通过 `pthread_cond_broadcast` 唤醒线程、执行任务，再通过 `cv_done` 等待所有工作线程完成，来大幅减少线程创建与销毁的开销。实际上，这一模式与 OpenMP 内部的线程池完全等效，只是对于 Pthread 来说需要我们来手动实现。

线程池核心调度逻辑（`thread_pool.h`）

```
1 void dispatch(std::function<void(int)> work) {
2     pthread_mutex_lock(&mu);
3     fn = work; pending = n; work_ver++;
4     pthread_cond_broadcast(&cv_work); // 唤醒所有工作线程
5     while (pending > 0)
6         pthread_cond_wait(&cv_done, &mu); // 等待全部完成
7     pthread_mutex_unlock(&mu);
8 }
```

(三) IVF (Inverted File Index) 算法

IVF 是一种基于聚类的近似最近邻搜索索引结构，其核心思想是将向量空间划分为若干个簇，查询时只扫描最相近的少数几个簇，从而大幅减少距离计算次数。

离线构建阶段：

对基向量进行 KMeans 聚类，得到 `nlist` 个簇心 $\{c_1, c_2, \dots, c_L\}$ ($L = \text{nlist}$)。每个基向量被分配到距离最近的簇：

$$j^*(v_i) = \arg \min_{j \in [L]} \|v_i - c_j\|^2$$

随后将所有 base 向量按其所属的簇进行**内存重排**，使得相同簇内的向量在内存中连续存储。这一操作可有效降低在线扫描时的 cache miss 率，是 IVF 性能的关键保障之一。

在线查询阶段：

- **粗排**：计算 query 与所有 `nlist` 个簇心的距离，选出最近的 `nprobe` 个簇。

$$d(q, c_j) = 1 - \langle q, c_j \rangle$$

$$\mathcal{P} = \text{top-}n\text{probe}\{d(q, c_j)\}_{j \in [L]}$$

在这里，`nprobe` 会同时影响精度与性能，是我们需要进行 trade-off 的主要参数之一，`nprobe` 越大召回率越高，但是相应地带来的延迟就越大。

- **精排**: 扫描所选 n_{probe} 个簇内的所有向量, 用 SIMD 加速的内积计算精确排序, 维护局部 top- k 优先队列。这部分与 SIMD 实验中的重排逻辑类似, 此处不再赘述。

多线程并行策略: 对于 IVF 来说, 粗排计算量较小 (仅 n_{list} 个簇心, 通常 ≤ 256), 并行化后线程创建与销毁带来的开销反而占主导, 多线程并行带来的收益过少, 在这部分我们保持单线程即可。在精排阶段中, 对 n_{probe} 个簇进行扫描, 是该算法主要的计算瓶颈, 这里我们采用**簇级动态并行**: 每个线程从共享任务队列中动态领取一个簇进行扫描, 完成后继续取下一个, 直到所有簇被处理完毕。这种动态调度策略能有效应对各簇大小不均的问题。

(四) IVF-PQ 算法

IVF-PQ 是将 IVF 与 PQ 的相结合的混合索引结构。在查询时, 利用 IVF 将候选范围缩减至 n_{probe} 个簇后, 再在候选簇内用 PQ 快速估算近似距离。从整体流程来看, 我们考虑两种思路, 分别实现了对应的方案:

- **全局 PQ**: 先对所有基向量 (按 IVF 重排后的顺序) 进行统一的 PQ 编码; 查询时在 n_{probe} 个候选簇内, 先用 ADC 查表得到近似距离, 筛选出 rerank_k 个候选, 再用原始浮点向量进行精排。
- **簇内独立 PQ**: 先构建 IVF 索引, 然后在簇内独立训练各簇自己的 PQ 量化器。由于每个簇内的向量分布更集中, PQ 量化误差更小, 召回率通常优于全局 PQ; 但代价是需要为每个簇分别存储 PQ 质心, 内存占用和索引构建开销更大。

(五) HNSW (Hierarchical Navigable Small World) 算法

HNSW 是一种基于图的近似最近邻索引结构, 其核心在于构建一个层次化的小世界图, 利用图的导航性质实现对数级别的搜索复杂度。在一个同时具有**高聚类系数**和**短平均路径长度**的图上, 贪心导航可以高效地逼近最近邻; HNSW 在此基础上引入层次结构, 通过指数衰减的节点采样在不同层构建具有较粗粒度的导航图, 使得单次查询的时间复杂度从线性降至对数级。本实验使用 `hnswlib` 库提供的索引, 在此基础上探索算法的原理及其多线程优化的并行策略。

索引结构: HNSW 构建一个包含 $L_{\max} + 1$ 层 ($l = 0, 1, \dots, L_{\max}$) 的层次图。第 l 层是一个近邻图 $G_l = (V_l, E_l)$, 其中 $V_0 = \{v_1, \dots, v_N\}$ 包含全部 N 个向量节点, 高层节点数按指数递减。每个节点 v 在第 l 层最多连接 M 个近邻边 (第 0 层最多连接 $2M$ 个), 这些边近似地指向该层中距离最近的若干节点。

层分配: 插入新节点 v 时, 其所在最高层 $\ell(v)$ 由以下公式随机采样:

$$\ell(v) = \lfloor -\ln(U(0, 1)) \cdot m_L \rfloor, \quad m_L = \frac{1}{\ln M}$$

其中 $U(0, 1)$ 为均匀分布随机数, m_L 为层级乘数。由此, 节点出现在第 l 层的概率为:

$$P(\ell(v) \geq l) = e^{-l/m_L}$$

即节点层级服从参数为 m_L 的指数分布, 高层节点数量呈指数衰减, 形成类似跳表的稀疏结构。

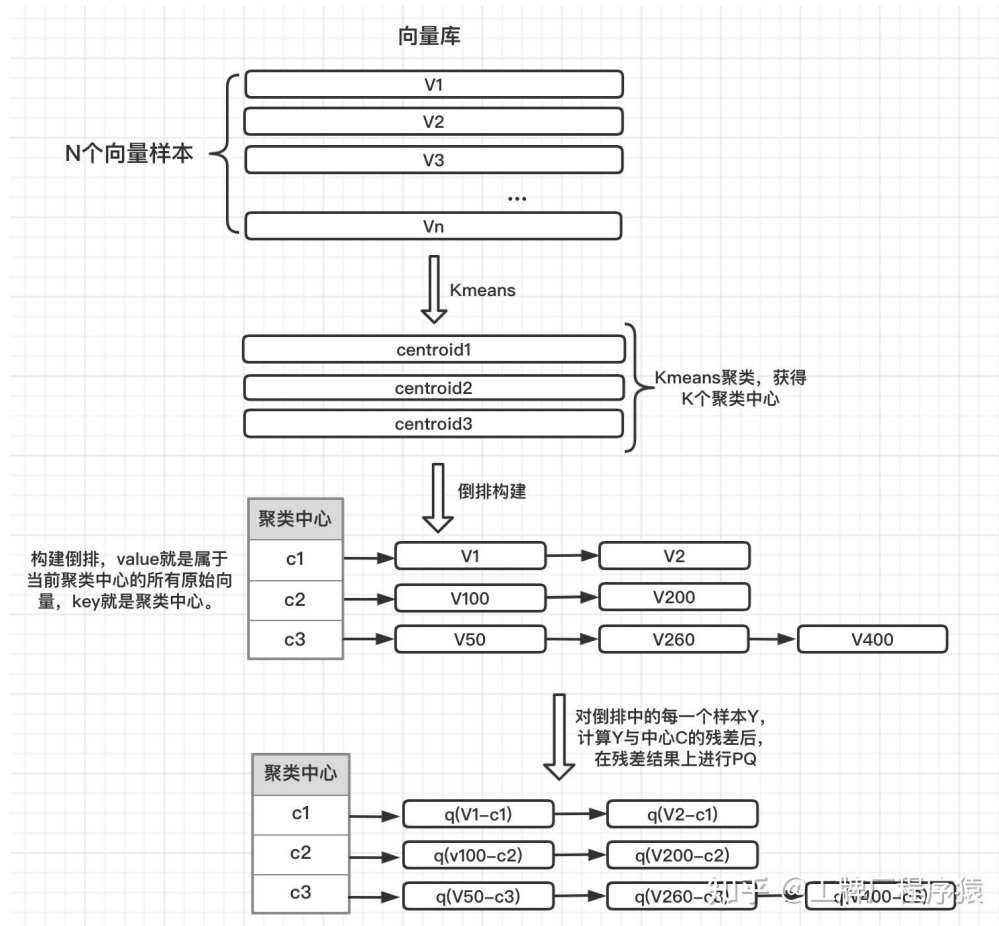


图 1: IVF-PQ 流程图-方法二 (簇内独立 PQ)

搜索算法： 查询向量 q 的搜索过程从最顶层 $L_{\max}$ 出发，逐层向下贪心导航。在每一层 $l > 0$ ，以当前最近节点为入口执行单次贪心搜索 ($ef = 1$)，将找到的最近点作为下一层的入口；在底层 ($l = 0$) 执行宽度为 ef_search 的集束搜索，维护大小为 ef_search 的候选集 $\mathcal{W}$ 。底层的集束搜索是决定召回率的核心步骤，其终止条件为：

$$d(q, c_{\min}) > d(q, w_{\max})$$

即候选集 $\mathcal{C}$ 中最近节点的距离已超过结果集 $\mathcal{W}$ 中最远节点的距离，此时继续探索不会再对结果有所提升，搜索停止。最终从 $\mathcal{W}$ 中取出前 k 个最近节点作为查询结果。

整个查询过程的时间复杂度为：

$$T_{\text{search}} = O(\log N)$$

参数 ef_search 的作用： ef_search (候选集大小) 是控制搜索精度与延迟的核心参数。较大的 ef_search 可以使算法探索更多的节点，召回率提升但是延迟增加，是后续实验进行参数分析的重点之一。

多子图并行搜索策略： 单个查询在 HNSW 底层图上的搜索过程依赖贪心路径，具有较强的串行依赖，直接并行化具有一定的难度。本实验采用**多子图并行**方案：将基向量划分为 T 个子集，分别构建 T 个独立的子图索引；每次查询时， T 个线程各自在其负责的子图上进行贪心搜索，最后再去归并各线程的 top- k 结果。该策略以较小的召回率开销换取显著的延迟降低，特别是在 ef_search 较大时优势明显。

三、 程序设计与实现

为了全面探讨各个算法的细节与特点，本实验一共实现了 S0~S18 共 19 个策略，按算法类型分组如下：

- S0~S2: Flat-SIMD 基线与多线程并行 (flat_thread.h)
- S3~S6: PQ-SIMD 基线与多线程并行 (pq_thread.h)
- S7~S9: IVF-SIMD 基线与多线程并行 (ivf.h)
- S10~S15: IVF-PQ 基线与多线程 (ivf_pq.h、ivf_pq_v2.h 两种方案)
- S16~S18: HNSW 进阶 (hnswh.h)

接下来将逐步讲解各个算法及其多线程编程的具体实现。

(一) Flat-SIMD 多线程编程 (S0~S2)

在本次实验，我们选取 S0 作为本次多线程实验的全局基线，即 lab2-SIMD 实验中已经实现的 Flat-SIMD 单线程版本。在实验框架下进行测试，得到结果与上次实验中的数据基本一致，延迟约 5761 μs ，召回率 0.99995。

在 S0 的基础上，我们开始本次实验的多线程编程。S1 (OpenMP) 和 S2 (Pthread) 依照基向量的下标进行静态分块实现并行。每个线程负责一段连续的区间，并在内部维护局部 top- k 优先队列，全部线程完成后将各个线程内的局部队列归并为全局 top- k ，即可得到最终结果。

Algorithm 1 Flat-SIMD 多线程并行 (S1/S2)**Input:** base 向量库 $Base[N]$, 查询向量 q , 维度 D , 线程数 T , 返回数量 k **Output:** 全局 top- k 结果队列 $final_q$

```

1: 将  $[0, N)$  均匀划分为  $T$  个连续分块  $\{[s_t, e_t)\}_{t=0}^{T-1}$ 
2: Parallel (各线程  $t$  独立执行):
3:   for  $i = s_t$  to  $e_t - 1$  do
4:      $ip \leftarrow \text{NEON\_IP}(q, Base[i])$  ▷ SIMD 加速内积
5:      $dis \leftarrow 1.0 - ip$ 
6:     用  $dis$  更新线程局部优先队列  $local\_qt$  (维持大小  $\leq k$ )
7:   end for
8: Barrier (等待所有线程完成)
9:  $final\_q \leftarrow \text{MergeTopK}(local\_q_0, local\_q_1, \dots, local\_q_{T-1}, k)$ 
10: return  $final\_q$ 

```

Pthread 版本 (S2) 逻辑类似, 但在实际的实现过程中, 我们发现线程管理是很重要一个问题。在最初的编写中, 我们直接调用了 Pthread 进行多线程编程, 但在测试时发现其延迟明显慢于 OpenMP, 在 $T=4$ 的场景下, S2 的平均延迟约为 S1 的 1.5~2 倍。

这一延迟实际上来自于线程调度带来的开销。在最开始的策略里, 每次查询时会经历创建线程-计算-销毁线程的过程, 这意味着线程会在循环内频繁地创建与销毁, 这无疑带来了许多不必要的开销。为了解决这一问题, 我们考虑在线程管理上进行改进, 采用线程池来维护线程的调用。

在具体实现上, 线程管理由 ThreadPool 接管, 通过 `pool.dispatch()` 分发任务, 从而消除 per-query 线程创建开销。经过测试, 修改后的 S2 明显恢复正常, 平均延迟与 OpenMP 相近。

线程池核心实现 (thread_pool.h)

```

1 // 工作线程: 在 cv_work 上休眠, 通过 work_ver 判断是否有新任务
2 static void* worker_loop(void* raw) {
3     WorkerArg* wa = (WorkerArg*)raw;
4     ThreadPool* p = wa->pool;
5     int tid = wa->tid, seen = 0;
6     while (true) {
7         pthread_mutex_lock(&p->mu);
8         while (p->work_ver == seen && !p->shutdown)
9             pthread_cond_wait(&p->cv_work, &p->mu); // 无任务时休眠, 不占用
              CPU
10        if (p->shutdown) { pthread_mutex_unlock(&p->mu); return nullptr; }
11        seen = p->work_ver;
12        auto f = p->fn;
13        pthread_mutex_unlock(&p->mu); // 释放锁后再执行, 不阻塞其他 worker
14        f(tid); // 执行本次查询分配的任务
15        pthread_mutex_lock(&p->mu);
16        if (--p->pending == 0)
17            pthread_cond_signal(&p->cv_done); // 最后完成的 worker 通知主线程
18        pthread_mutex_unlock(&p->mu);
19    }
20 }
21 // 主线程调用: 分发任务并阻塞等待所有 worker 完成

```



```

22 void dispatch(std::function<void(int)> work) {
23     pthread_mutex_lock(&mu);
24     fn = work; pending = n; work_ver++; // 递增版本号, 触发 worker 唤醒
25     pthread_cond_broadcast(&cv_work); // 广播唤醒所有休眠 worker
26     while (pending > 0)
27         pthread_cond_wait(&cv_done, &mu); // 主线程阻塞, 直到全部完成
28     pthread_mutex_unlock(&mu);
29 }

```

在这部分, 由于 Flat 扫描属于计算密集型任务, 基向量均匀分块后负载天然均衡, 静态调度已经可以实现接近线性的加速比。但是, 受制于硬件层面真实的物理核数, 当线程过多时 ($T > 8$), 依然存在加速瓶颈, 甚至出现负优化的情况。

(二) PQ-SIMD 多线程编程 (S3~S6)

在上次的 SIMD 实验中, 我们已经实现了 PQ-Rerank-SIMD 这一算法, 可以直接作为本次实验的基线。但我们在上一次实验中, 其实还完成了更进一步的优化, 实现了跨质心的并行以及较为基础的 OPM 优化。两者在计算量较大时具有明显差异 (详见 lab2)。为了保持实验的连贯性, 这里我们将 PQ-Final 还原为串行版本, 以实现了 SoA 与跨质心并行的 PQ 作为这部分的基线, 并在其基础上, 进一步引入多线程编程进行优化。

S3(PQ-Rerank-SIMD)采用 AoS 编码布局, 单线程串行扫描全库, LUT 构建按子空间逐个计算, 和在 SIMD 实验中实现的基本一致。**S4**(PQ-Final)则采用 SoA 编码布局(pq_codes_soa[M][N])和 SoA 质心布局 (pq_centroids_soa[M][sub_dim][K]), LUT 构建使用跨质心 4 路 SIMD 并行, 扫描阶段通过循环交换实现连续内存访问。这里我们使用 S4 作为 S5/S6 多线程优化的基准。

在前一小结, 我们已经深入探讨了线程池的重要性, 在后续的编程中, 两种多线程编程的实现逻辑基本趋于一致, 在 Pthread 部分, 将默认使用我们手动构建的线程池进行线程调度。在这部分, **S5** (OpenMP) 和 **S6** (Pthread) 参考 S4 的基础流程, 主要有以下三阶段的并行流水线:

1. **LUT 构建**: 以子空间 m 为粒度进行外层并行 ($M = 8$ 个子空间分配给各线程), 内层仍用跨质心 SIMD 加速。
2. **ADC 粗排 + 线程内精排**: 对基向量按线程数进行静态分块, 每线程独立维护局部的粗排队列 (大小 rerank_k); 扫描完自己分块后立即在线程内触发精排, 无需任何锁或原子操作。
3. **全局归并**: 汇总各线程的局部 top-k 精排结果, 合并为最终 top-k。

Algorithm 2 PQ-Final 多线程并行流水线 (S5/S6)

Input: q , SoA 编码表 $Codes[M][N]$, SoA 质心 $Cents[M][SubDim][K]$, k , rerank_k, 线程数 T

Output: 全局 top-k 结果队列 $final_q$

- 1: ▷ 阶段 1: LUT 构建 (子空间级并行)
- 2: **Parallel for** $m = 0$ to $M - 1$ (每线程负责若干子空间):
- 3: **for** $c = 0$ to $K - 1$ **step** 4 **do**
- 4: $LUT[m][c:c+3] \leftarrow SIMD4_IP(q_m, Cents[m][\cdot][c:c+3])$ ▷ 跨质心 4 路 SIMD
- 5: **end for**
- 6: **Barrier** (等待 LUT 完整构建)
- 7: ▷ 阶段 2: ADC 粗排 + 线程内精排 (向量级并行)

```

8: 将  $[0, N)$  均匀划分为  $T$  块; Parallel (各线程  $t$  独立执行):
9: for  $i \in$  线程 $t$  负责的块 do
10:    $ip \leftarrow \sum_{m=0}^{M-1} \text{LUT}[m][\text{Codes}[m][i]]$  ▷ SoA 连续访存
11:   更新  $\text{local\_coarse\_}q_t$  (维持大小  $\leq \text{rerank\_k}$ )
12: end for
13: while  $\text{local\_coarse\_}q_t \neq \emptyset$  do
14:    $idx \leftarrow \text{local\_coarse\_}q_t.\text{pop}().\text{index}$ 
15:    $\text{exact\_dis} \leftarrow 1 - \text{NEON\_IP}(q, \text{Base}[idx])$ 
16:   更新  $\text{local\_fine\_}q_t$  (维持大小  $\leq k$ )
17: end while
18: ▷ 阶段 3: 全局归并
19:  $\text{final\_}q \leftarrow \text{MergeTopK}(\text{local\_fine\_}q_0, \dots, \text{local\_fine\_}q_{T-1}, k)$ 
20: return  $\text{final\_}q$ 

```

Pthread 版本 (S6) 在 LUT 构建阶段采用 `pthread_barrier_t` 显式同步: 仅由 `tid=0` 的线程完成 LUT 构建, 其余线程在 `barrier` 处等待, 随后所有线程并行进入 ADC 扫描阶段。

值得注意的是, 在测试时发现, 随线程数提升召回率也会相应增加。这一现象我们其实已经在上一轮的实验中进行了解释, 这里简单地回顾一下。当单线程时, 只有 `rerank_k` 个候选进入重排; 引入多线程后, 每个线程各自从 N/T 的子集中取出 `rerank_k` 个候选, 从全局来看, 实际的候选集具有 $T \times \text{rerank_k}$ 个独立样本, 且其分布也比单线程更加均匀, 这将使精排时的候选集多样性增加, 从而大幅提升了最终的召回率。

(三) IVF-SIMD 多线程编程 (S7~S9)

S7 (IVF-SIMD baseline) 为单线程 IVF 实现, 依然分为粗排和精排两步, 是 S8/S9 进行多线程优化的基础。

Algorithm 3 IVF 多线程精排 (Pthread 动态调度, ivf.h)

Input: query, vecdim, k , `probe_list[nprobe]`, 共享计数器 `next_task_idx`, 互斥锁 `mtx`

Output: 线程局部 top- k 优先队列 q

```

1: while true do
2:   lock(mtx)
3:    $\text{task} \leftarrow \text{next\_task\_idx}++$ 
4:   unlock(mtx)
5:   if  $\text{task} \geq \text{nprobe}$  then break
6:   end if
7:    $c \leftarrow \text{probe\_list}[\text{task}]$ 
8:   for  $i = \text{cluster\_starts}[c]$  to  $\text{cluster\_starts}[c+1] - 1$  do
9:      $ip \leftarrow \text{NEON\_IP}(\text{query}, \text{reordered\_base}[i])$ 
10:     $\text{dis} \leftarrow 1 - ip$ 
11:    if  $q.\text{size}() < k$  or  $\text{dis} < q.\text{top}()$  then
12:       $q.\text{push}(\{\text{dis}, \text{reordered\_ids}[i]\})$ 
13:      if  $q.\text{size}() > k$  then  $q.\text{pop}()$ 
14:    end if
15:  end if

```

```

16:   end for
17: end while

```

S8 (OpenMP) 使用 `#pragma omp for schedule(dynamic, 1) nowait` 对 `probe_list` 中的簇进行动态调度。每个线程维护独立的局部 `top-k`，全部簇处理完毕后再进行统一的归并。

S8 OpenMP 动态调度精排 (ivf.h)

```

1 #pragma omp parallel num_threads(nthr)
2 {
3     int tid = omp_get_thread_num();
4     auto& q = local_q[tid];
5     // nowait: 各线程完成自己分配的簇后不等待，直接进入归并
6     #pragma omp for schedule(dynamic, 1) nowait
7     for (int pi = 0; pi < (int)probe_list.size(); ++pi) {
8         int c = probe_list[pi];
9         for (size_t i = cluster_starts[c]; i < cluster_starts[c+1]; ++i) {
10             float ip = ivf_neon_ip(query, reordered_base + i*vecdim, vecdim);
11             float dis = 1.0f - ip;
12             if (q.size() < k || dis < q.top().first) {
13                 q.push({dis, reordered_ids[i]});
14                 if (q.size() > k) q.pop();
15             }
16         }
17     }
18 }
19 // 归并 local_q[0..nthr-1] -> final_q

```

S9 (Pthread) 则需要我们手动实现线程的动态调度：通过共享整型计数器 `next_task_idx` 和互斥锁，模拟线程从任务队列中动态领取簇的行为。

Algorithm 4 IVF Pthread 动态调度精排 (S9, ivf.h)

Input: query, k , `probe_list[nprobe]`, 共享计数器 `next_idx` (初始为 0), 互斥锁 `mtx`

Output: 线程局部 `top-k` 优先队列 q

```

1: while true do
2:   lock(mtx)
3:   task ← next_idx++
4:   unlock(mtx)
5:   if task ≥ nprobe then break
6:   end if
7:   c ← probe_list[task]
8:   for i = cluster_starts[c] to cluster_starts[c+1] - 1 do
9:     ip ← NEON_IP(query, reordered_base[i])
10:    更新 q (维持大小 ≤ k)
11:   end for
12: end while

```

这种动态调度的优势在于，当 `nprobe` 较少（如 16）而线程数较多（如 $T=8$ ）时，OpenMP

的静态划分会导致部分线程出现空闲的情况，而 Pthread 的动态调度仍能保证每个线程都有较为活跃的任务，从而在特定参数场景下具有一定的优势。

(四) IVF-PQ (S10~S15)

IVF-PQ 在 IVF 粗筛基础上，用 PQ 的 ADC 查表替代精确内积计算，实现进一步加速。本实验实现两种方案，其在线查询流程存在明显差异。

方法一 (S10~S12): 全局 PQ 全库共享同一套 PQ 量化器，LUT 构建也只需一次，而后在所有候选簇上复用同一张 LUT 进行查表。

Algorithm 5 IVF-PQ 方法一在线查询 (S10 单线程, S11/S12 多线程)

Input: $q, nprobe, rerank_k, k$

Output: top- k 结果

```

1:                                     ▷ 步骤 1: IVF 粗排 (单线程)
2:  $\mathcal{P} \leftarrow \text{top-}nprobe \{d(q, c_j)\}_{j=1}^L$ 
3:                                     ▷ 步骤 2: LUT 构建 (仅一次, 全局共享)
4: for  $m = 0$  to  $M - 1$  do
5:   for  $c = 0$  to  $K - 1$  do
6:      $LUT[m][c] \leftarrow \langle q_m, \mu_m^c \rangle$ 
7:   end for
8: end for
9:                                     ▷ 步骤 3: ADC 粗排 (多线程并行, 按候选簇动态分配)
10: Parallel for  $j \in \mathcal{P}$  (S11 OMP dynamic / S12 Pthread 共享计数器):
11:   for  $v_i \in \text{簇 } j$  do
12:      $\tilde{d}(q, v_i) \leftarrow 1 - \sum_{m=0}^{M-1} LUT[m][b_m^{(i)}]$ 
13:     更新线程局部  $coarse\_q$  (大小  $\leq rerank\_k$ )
14:   end for
15: 归并各线程  $coarse\_q$  得全局 top- $rerank\_k$  候选
16:                                     ▷ 步骤 4: 精排
17: for  $idx \in \text{top-}rerank\_k \text{ 候选}$  do
18:    $d_{\text{exact}} \leftarrow 1 - \text{NEON\_IP}(q, \text{Base}[idx])$ 
19:   更新  $fine\_q$  (维持大小  $\leq k$ )
20: end for
21: return  $fine\_q$ 

```

方法二 (S13~S15): 簇内独立 PQ 每个簇有独立的 PQ 量化器，LUT 需对每个候选簇分别构建。其在线查询流程在 LUT 构建与 ADC 粗排阶段有本质差异：

Algorithm 6 IVF-PQ 方法二在线查询 (S13 单线程, S14/S15 多线程)

Input: $q, nprobe, rerank_k, k$, 每簇独立量化器 $\{PQ_j\}$

Output: top- k 结果

```

1:                                     ▷ 步骤 1: IVF 粗排 (同方法一)
2:  $\mathcal{P} \leftarrow \text{top-}nprobe \{d(q, c_j)\}_{j=1}^L$ 
3:                                     ▷ 步骤 2~3: 按候选簇循环, 逐簇构建 LUT 后 ADC (多线程并行)
4: Parallel for  $j \in \mathcal{P}$  (S14 OMP dynamic / S15 Pthread 共享计数器):
5:   for  $m = 0$  to  $M - 1$  do

```

```

6:   for  $c = 0$  to  $K - 1$  do
7:        $\text{LUT}_j[m][c] \leftarrow \langle (\tilde{q}_j)_m, (\mu_j)_m^c \rangle$  ▷ 使用簇  $j$  自己的质心
8:   end for
9: end for
10: for  $v_i \in$  簇  $j$  (已用  $\text{PQ}_j$  编码) do
11:      $\tilde{d}(\tilde{q}_j, r_i) \leftarrow 1 - \sum_{m=0}^{M-1} \text{LUT}_j[m][b_m^{(i)}]$ 
12:     更新线程局部  $\text{coarse}_q$ 
13: end for
14: 归并  $\rightarrow$  精排 (同方法一步骤 4)
15: return  $\text{fine}_q$ 

```

在方法二中,每个候选簇都需要独立构建一次 LUT, LUT 构建带来总开销是方法一的 n_{probe} 倍, 这是方法二单线程延迟更高的直接原因; 但其召回率通常优于方法一。

(五) HNSW 进阶 (S16~S18)

在实验框架中, 已经提供了 hnsplib, 我们在 S16 (HNSW 单线程基线) 中直接调用 hnsplib 的标准搜索接口即可。实际调用中还需设置 `ef_search` 参数控制集束搜索的候选集大小, 进而权衡召回率与延迟。

S17 (OpenMP) 和 S18 (Pthread) 均采用多子图并行策略, 其完整实现流程如下:

Algorithm 7 HNSW 多子图并行搜索 (S17/S18)

Input: 查询 q, T 个独立子图 $\{G_t\}_{t=1}^T$ (各覆盖 N/T 个向量), $k, \text{ef}=64$

Output: 全局 top- k 结果 R

```

1: ▷ 并行阶段: 各线程在各自子图上独立贪心搜索
2: Parallel for  $t = 1$  to  $T$  (S17 OpenMP / S18 Pthread 线程池):
3:     从  $G_t$  的入口点  $ep_t$  出发, 执行 HNSW 贪心搜索:
4:         自顶层逐层导航, 每层取当前最近点作为下一层入口
5:         底层执行 ef-集束搜索, 维护候选集  $\mathcal{W}_t$  (大小  $\leq \text{ef}$ ):
6:             终止条件:  $d(q, c_{\min}) > d(q, w_{\max})$ 
7:          $R_t \leftarrow \text{top-}k \mathcal{W}_t$  ▷ 线程  $t$  的局部 top- $k$ 
8: ▷ 归并阶段 (主线程)
9:  $R \leftarrow \text{top-}k \bigcup_{t=1}^T R_t$ 
10: return  $R$ 

```

S17 OpenMP 多子图搜索 (hnsplib)

```

1 std::vector<std::priority_queue<std::pair<float, uint32_t>>> local_res(nthr);
2 #pragma omp parallel for num_threads(nthr) schedule(static, 1)
3 for (int t = 0; t < nthr; ++t) {
4     // 在子图 t 上执行 hnsplib 标准搜索 (ef=64)
5     auto knn = sub_indices[t] -> searchKnn(query, k);
6     while (!knn.empty()) {
7         local_res[t].push({knn.top().first, knn.top().second});
8         knn.pop();
9     }
10 }

```

```
11 // 归并 local_res[0..nthr-1] -> final top-k
```

多子图策略的核心优势在于: 每个子图使用较小的 $ef=64$, 单次搜索路径短; 但 T 个线程集成后能覆盖更大范围, 使得在与 S16 同等召回率下, 整体延迟显著降低。

(六) 离线索引构建的并行策略

在实验过程中, 我们也发现, 很多策略在离线的索引构建阶段 (KMeans 聚类、向量编码、内存重排等) 也有一定的多线程优化空间。由于索引构建的时间不参与查询延迟的统计, 这里我们仅对其进行简单的并行优化。

所有策略的离线索引构建阶段统一使用 **OpenMP 4 线程**完成, 与在线查询的并行方式相互独立, 确保各策略的在线测试在相同的索引质量下进行, 减少额外变量的引入。

实际上, 虽然离线阶段的优化无法减少查询的平均延迟, 但它能让我们程序运行的总时长减少, 这在本次实验流程中也是很重要的。由于涉及的策略算法较多, 且具有多种参数需要分析讨论, 这部分的优化可以大大加快我们进行后续测试的效率。

四、 实验结果分析

(一) 数据总览

本实验在 DEEP100K 数据集上对全部 19 个策略进行测试 (2000 次查询, $k=10$)。

在测试过程中, 由于服务器多人共享, 实际性能存在一定程度的波动。为了让我们的数据尽可能地接近真实性能作为标准, 我们采取多次测量取最小值的方法, 并在凌晨使用人数较少时进行集中的测试。同时, 由于策略较多, 参数较为复杂, 我们还借助 AI 大模型, 改写了一版便于使用脚本批量测试的 main 以及对应的测试脚本 run_batch.py (详见代码仓库)。借助这两项工具, 我们可以更加方便、更加精确地得出测试结果。

经过测试整理数据, 主要有以下各个算法具有代表性的测试结果。这里我们以 S0(Flat-SIMD 单线程, $5761.17 \mu s$) 作为全局基线进行比较。

表 1: 实验数据总览

策略编号	算法名称	线程数	召回率	延迟 (μ s)	加速比 (vs S0)
S0	Flat-SIMD 单线程基线	—	0.99995	5761.17	1.00×
S1	Flat-SIMD OpenMP	4	0.99995	1587.22	3.63×
S2	Flat-SIMD Pthread	4	0.99995	1987.00	2.90×
S3	PQ-SIMD(AoS) 单线程基线 ¹	—	0.9461	1475.09	3.91×
S4	PQ-SIMD(SoA) 单线程基线	—	0.9461	1486.28	3.88×
S5	PQ-SIMD OpenMP	4	0.9950	585.11	9.85×
S6	PQ-SIMD Pthread	4	0.9950	667.63	8.63×
S7	IVF 单线程基线 (nprobe=16) ²	—	0.9583	336.28	17.13×
S7	IVF 单线程基线 (nprobe=32)	—	0.9876	645.48	8.93×
S8	IVF OpenMP(nprobe=16)	4	0.9583	136.95	42.07×
S9	IVF Pthread(nprobe=16)	4	0.9583	176.61	32.62×
S10	IVF-PQ-M1 单线程基线 ³	—	0.9107	454.86	12.67×
S11	IVF-PQ-M1 OpenMP	4	0.9537	259.78	22.18×
S12	IVF-PQ-M1 Pthread	4	0.9536	315.29	18.27×
S13	IVF-PQ-M2 单线程基线	—	0.9376	781.17	7.38×
S14	IVF-PQ-M2 OpenMP	4	0.9574	380.13	15.16×
S15	IVF-PQ-M2 Pthread	4	0.9574	416.22	13.84×
S16	HNSW 单线程基线 (ef=32) ⁴	—	0.9459	193.91	29.71×
S16	HNSW 单线程基线 (ef=64)	—	0.9808	315.70	18.25×
S17	HNSW OpenMP(ef=64)	4	0.9963	288.18	19.99×
S18	HNSW Pthread(ef=64)	4	0.9963	336.50	17.12×

¹ PQ-SIMD 系列实验取 rerank_k=500; ² IVF 系列实验取 nlist=256; ³ IVF-PQ 系列实验取 nlist=256,nprobe=16,rerank_k=500; ⁴ HNSW 系列实验取 M=16,ef_construction=200。

总体来看, 各项策略的优化效果较为显著, 使用更优的策略与多线程的并行优化可以带来明显的效率提升, 且均能维持在较高的精度水平。其中 **IVF + OpenMP T=4** 取得了 42.07× 的优异成绩, T=8 时更是直接来到了 52.91× 的极致加速比, 相比 Lab 2 的 PQ-Final(约 15×) 有了进一步的飞跃; 同时, HNSW 多子图并行也以高精度的召回率与优异的加速效果展示了图索引的潜力。

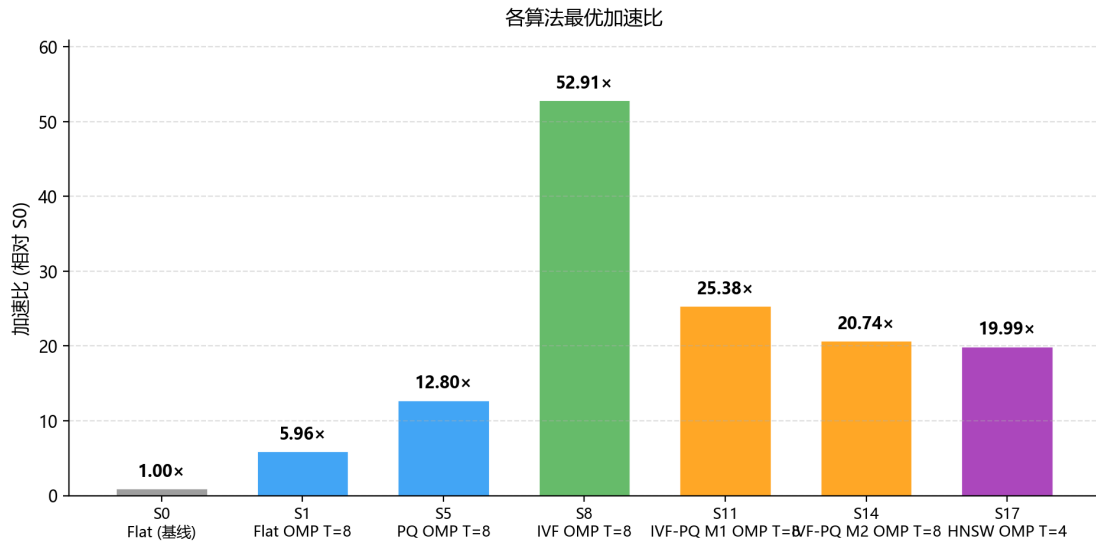


图 2: 各算法最优策略加速比对比

考虑到线程数与策略中相应的参数较为复杂，会同时影响召回率与平均延迟，接下来我们将对各项参数进行进一步的深入分析，分析各个参数的影响，并对参数的权衡进行探讨。

(二) 线程数分析

在实际的运算中，多线程的扩展性并非随线程数线性增长，在实际工程中，主要受到以下几方面因素的制约：

- **物理核数限制：**实验所用 ARM 服务器具有 8 个物理核心，当 $T \leq 8$ 时每个线程可独占一个物理核， $T > 8$ 时多个线程会被调度到同一物理核上分时执行，加速比难以继续增长。
- **并行任务粒度：**当任务粒度过小（如 IVF 在 nprobe 较少时，单簇扫描时间仅几微秒），线程调度与同步开销可能与有效计算量相当，从而抵消并行带来的收益。
- **内存带宽瓶颈：**对于 IVF-PQ 这类访存密集型算法，多线程同时访问 LUT 与编码表，L2/L3 cache 与内存带宽逐渐成为瓶颈，此时加速比会显著低于线性。
- **同步与归并开销：**每次查询结束需要归并各线程的局部 top-k，这部分开销是固定的串行成本，在低延迟场景下占比相对增加。

表 2: 各算法在不同线程数下的延迟 (μs) 与最佳加速比

算法 (并行方式)	T=1	T=2	T=4	T=8	单线程基线	最优加速比 (vs T=1)
Flat OMP (S1)	5940.0	2961.6	1587.2	967.1	5761.2	6.14 $\times$ (T=8)
Flat Pthread (S2)	6061.0	3063.3	1987.0	1561.4	5761.2	3.88 $\times$ (T=8)
PQ OMP (S5)	1491.6	857.9	585.1	450.2	1486.3	3.31 $\times$ (T=8)
PQ Pthread (S6)	1462.3	917.9	667.6	738.0	1486.3	2.19 $\times$ (T=4)
IVF OMP (S8)	312.9	180.7	136.9	108.9	336.3	2.87 $\times$ (T=8)
IVF Pthread (S9)	357.0	219.7	176.6	181.0	336.3	2.02 $\times$ (T=4)
IVF-PQ M1 OMP (S11)	437.1	326.2	259.8	227.0	454.9	1.93 $\times$ (T=8)
IVF-PQ M1 Pthread (S12)	446.2	355.4	315.3	316.4	454.9	1.42 $\times$ (T=4)
IVF-PQ M2 OMP(S14)	736.1	521.2	380.1	277.8	781.2	2.65 $\times$ (T=8)
IVF-PQ M2 Pthread(S15)	815.7	616.0	416.2	361.8	781.2	2.25 $\times$ (T=8)

从表 2 中可以归纳出几个清晰的规律:

- **OpenMP 在线程数较多时整体扩展性优于 Pthread:** 几乎所有算法的 OMP 版本在 T=8 时仍能继续降低延迟, 而 Pthread 版本多数已出现停滞甚至负优化的情况 (如 S6 PQ Pthread, T=8 的 738 μs 反而高于 T=4 的 668 μs)。这一现象的原因在于我们手动实现的 Pthread 动态调度依赖互斥锁, 在线程数较多时带来了显著的锁竞争开销。
- **Flat 等计算密集型任务接近理想扩展:** Flat OMP 在 T=4 下取得 3.74 $\times$ 、T=8 下取得 6.14 $\times$ 的算法内加速比 (相对 T=1 单线程同算法版本), 虽未达到理论线性, 但其效果已经接近实际上限。
- **IVF-PQ 多线程边际收益最低:** M1 OMP 从 T=1 的 437 μs 提升到 T=8 仅至 227 μs , 算法内比较的加速比仅 1.93 $\times$, 这与其访存密集型本质相符, ADC 查表对内存带宽的依赖在多线程下成为了优化的瓶颈。

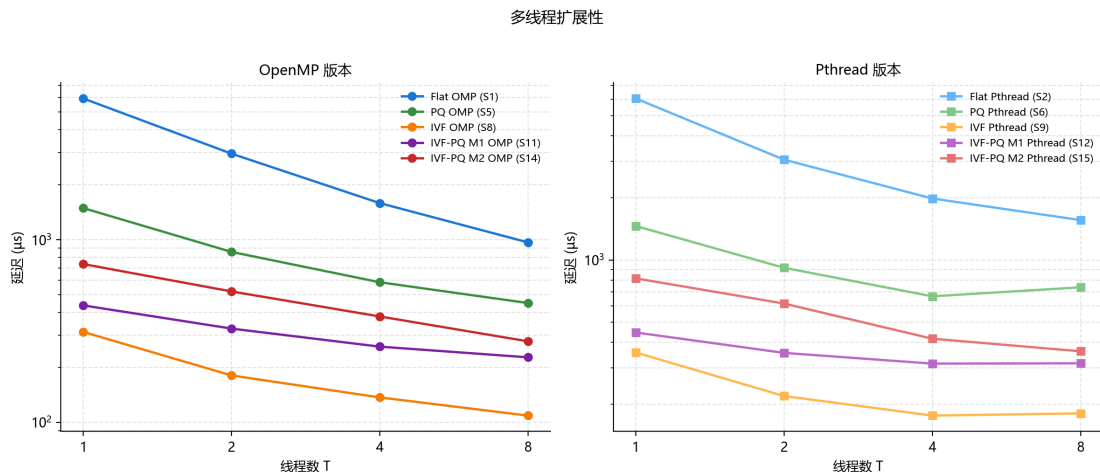


图 3: 多线程扩展性折线图

实际上, 本次实验主要受制于问题规模与实验环境的物理核数。实验中其实也测试了 16 线程下的优化效果, 但毫无例外地均呈现出负优化的结果。一方面, 实验数据集并非大数量级的工业化场景, 线程开销会成为不可忽视的一大问题; 另一方面, 虽然存在超出物理核数仍能取得优化效果的案例 (如超线程、同步多线程等), 但在本实验的服务器环境中往往难以精准复现, 测量时的数据波动已经可以覆盖其优化的效果; 即便或许在统计上确实有所提升, 但考虑多方面的因素, 其带来的实际收益微乎其微。这也体现出我们在多线程并行策略下的优化边界。

(三) 参数权衡

除了线程数之外, 各算法自身的参数 (如 `rerank_k`、`nprobe`、`nlist`、`ef_search` 等) 也会带来复杂的效果, 同时影响召回率与延迟。关于 PQ 的 `rerank_k` 已在 Lab 2 中详细讨论 (其作用在于扩大精排候选池来弥补量化误差), 在本次实验中不再赘述。这里我们主要聚焦 IVF/IVF-PQ 的 `nprobe` 和 HNSW 的 `ef_search` 这两个新引入的参数。

1. IVF / IVF-PQ: `nprobe`

在 IVF 的算法中, `nprobe` 控制粗排阶段保留的候选簇数。其值越大, 精排阶段需要扫描的向量就越多, 延迟越高但召回率也相应提升。我们对 IVF (S7) 和 IVF-PQ 两种方法 (S10/S13) 在 `nprobe=4,8,16,32,64` 下进行了系统的测试。

表 3: IVF / IVF-PQ nprobe 参数扫描 (单线程基线)

nprobe	S7 (IVF)		S10 (IVF-PQ M1)		S13 (IVF-PQ M2)	
	召回率	延迟 (μs)	召回率	延迟 (μs)	召回率	延迟 (μs)
4	0.8002	92.22	0.7836	255.67	0.7945	347.17
8	0.8986	174.50	0.8654	348.83	0.8847	495.80
16	0.9583	336.28	0.9107	454.86	0.9376	781.17
32	0.9876	645.48	0.9311	631.70	0.9601	1197.79
64	0.9974	1272.84	0.9359	840.20	0.9668	1893.51

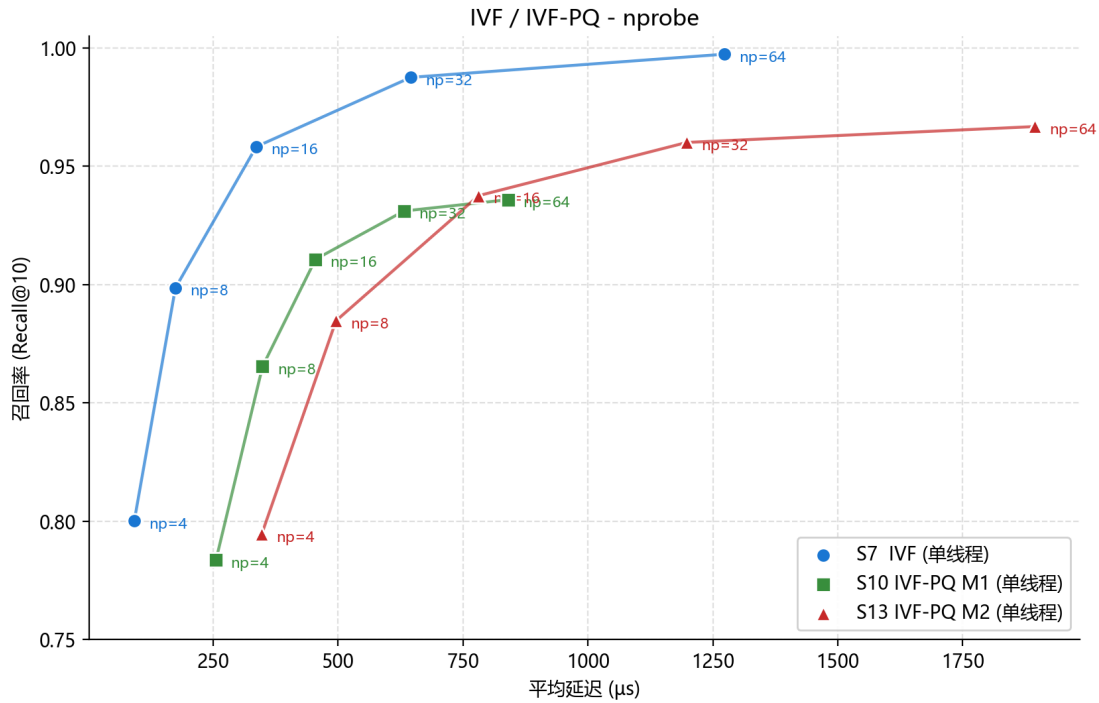


图 4: IVF / IVF-PQ 的 recall-latency 散点图

从数据中我们可以看出:

- IVF (S7) 单线程在 nprobe 较小时性价比最高, nprobe=16 是召回率与延迟的一个重要分水岭, nprobe 从 4 增至 16 时召回率快速从 0.80 提升至 0.96, 平均延迟仅增加 244 μs ; 而再从 16 增至 64, 带来的召回率提升迅速放缓, 而延迟却提高近 940 μs 。
- IVF-PQ 两种方法的曲线均位于 IVF 下方 (同 latency 下召回率更低), 说明 PQ 量化引入了额外的误差。在相同召回率 0.93 附近, IVF-PQ M1 (nprobe=32) 延迟为 632 μs , 而 IVF-PQ M2 (nprobe=16) 需要 781 μs , 明显高于纯 IVF 的延迟。
- 在图像中, 方法二的曲线始终高于方法一, 这印证了我们前面簇内局部 PQ 量化精度更高的理论预期; 方法二在算法精度上显然更优, 但相应地, 也意味着更高的延迟代价 (单线

程下 $nprobe=64$ 时延迟约为方法一的 2.3 倍)。这两种方法各有所长，后面会进行更加具体的分析。

此外，和 IVF 相关的还有 $nlist$ (总簇数) 这一参数，也会影响单个簇内的平均向量数。通过计算我们可以知道，在 $base_number = 100K$ 、 $nlist = 256$ 时，平均每簇约 390 个向量；若 $nlist$ 增至 1024 则平均每簇约 98 个向量，精排扫描更快但簇心比较开销随之增加。但经过实际测试我们发现， $nlist$ 带来的影响远远不及 $nprobe$ 对于算法的决定性作用，在本实验中，我们使用推荐的固定值 $nlist=256$ 来进行其他的测试，这一取值在该数据规模下已经有足够良好的表现。

2. HNSW: ef_search

ef_search (集束搜索宽度) 是 HNSW 单线程版本 (S16) 的核心调节参数。S17/S18 由于采用多子图策略，当每个子图使用 $ef=64$ 时，召回率已经可以稳定在 99.63% 这一较高水平，所以这里我们主要观察单线程版本的数据进行分析。

表 4: HNSW ef_search 参数扫描

ef_search	S16 (单线程)	
	召回率	延迟 (μs)
16	0.8708	122.06
32	0.9459	193.91
64	0.9808	315.70
128	0.9952	519.43
200	0.9979	767.35

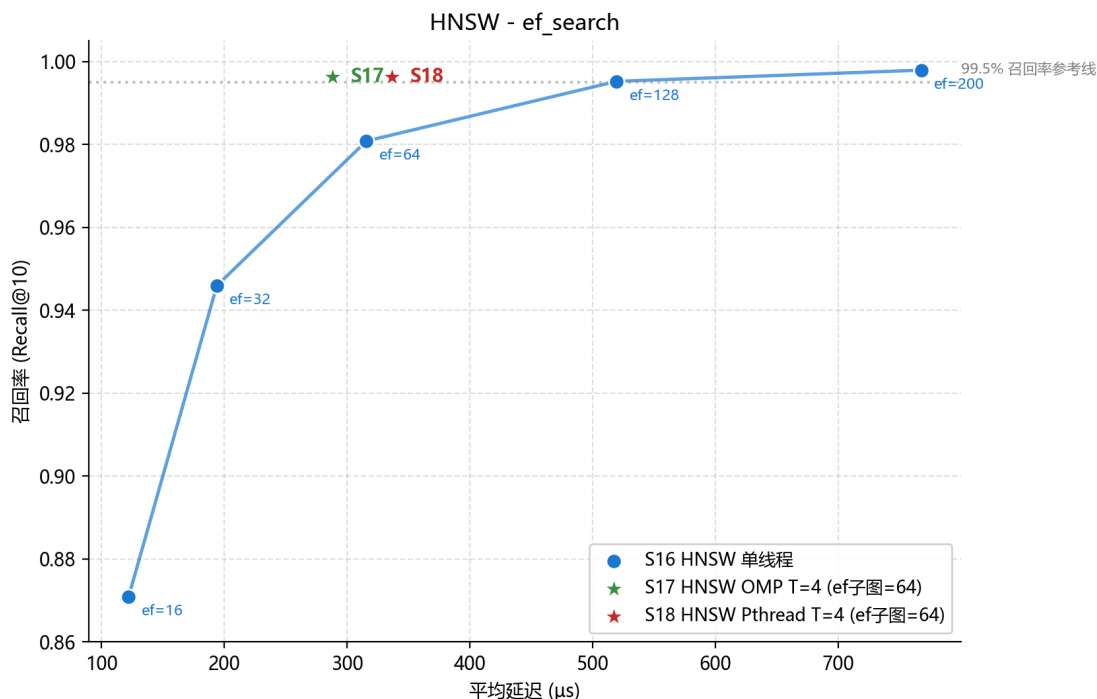


图 5: HNSW 的 recall-latency 散点图

分析上述数据我们可以看出，S16 单线程的 recall-latency 曲线呈现典型的边际递减形态，ef=128 时召回率达 0.995，继续增至 ef=200 仅再提升 0.003，而延迟则是从 519 μ s 提升到了 767 μ s。

实际上，由于我们使用了多子图并行的策略，考虑算法的总计算量，对于 T 线程 $ef = ef_t$ ，对应的其实是单线程的 $ef = ef_t \times T$ ；而除了计算量的差异，多子图带来的广度提升也在很大程度上提升了召回率。虽然单个子图找得不够深，但 T 个线程联合起来可以覆盖更加广阔的空间，用空间多样性弥补了单图深度的不足。实验结果也证明了：在达到相同的高水平召回率时，多子图并行（多个小 ef）的整体延迟，往往远低于单线程（单个大 ef）。

(四) perf 分析

为从硬件层面进一步分析多线程算法的瓶颈特征，我们使用 perf stat 工具对两类有代表性的策略进行了对比测试：S8 (IVF + OpenMP T=4) 与 S11 (IVF-PQ M1 + OpenMP T=4)，两者均使用 nprobe=16。运行命令如下：

perf 测试命令

```

1 # S8: IVF + OMP 4 线程
2 perf stat -e cache-references,cache-misses,cycles,instructions,context-
  switches \
3   ./main --strategy 8 --nprobe 16 --omp 4
4
5 # S11: IVF-PQ M1 + OMP 4 线程
6 perf stat -e cache-references,cache-misses,cycles,instructions,context-
  switches \
7   ./main --strategy 11 --nprobe 16 --rerank_k 500 --omp 4

```

测试结果如下：

表 5: perf 性能指标实测对比 (S8 vs S11, T=4, nprobe=16)

指标	S8 (IVF OMP)	S11 (IVF-PQ M1 OMP)
缓存引用数	6,339,569,906	13,892,465,830
缓存缺失数	76,360,103	100,616,534
缓存缺失率	1.204%	0.724%
周期数	12,797,735,166	29,245,176,898
指令数	24,477,706,583	67,108,457,103
IPC	1.91	2.29
总执行时间	1.34 s	6.21 s
用户时间	4.98 s	11.41 s
系统时间	0.04 s	0.05 s
平均查询延迟	163.68 μ s	307.58 μ s

观察实测数据，我们进行以下深入分析。

(1) 缓存缺失率 S8 高于 S11 (1.20% / 0.72%)

直觉来看，IVF-PQ 的 LUT 随机访问会引发更多 cache miss，但实际情况反而相反。原因其实在于两者的访存对象不同：

- S8 在精排阶段扫描的是内存重排后的 **float 基向量** (128 维 $\times$ 4 字节 = 512 字节/向量)，扫描 nprobe=16 个簇总计约 6400 个向量，需要传输约 3.2 MB 数据；
- S11 在 ADC 粗排阶段扫描的是 **uint8 PQ 编码** (每向量仅 8 字节，为原始 float 的 1/64)，全库 100K 向量的编码总大小约 800 KB，可以大部分驻留在 L3 cache 中；同时 LUT 只有 $M \times K = 8 \times 256 = 2048$ 个浮点数 (8 KB)，始终热在 L1 cache 里。

因此，S11 虽然绝对缺失数略多 (100M vs 76M)，但大幅度的数据压缩使其缓存利用率实际上更高，从而又更低的 Cache Miss。

(2) IPC S11 高于 S8 (2.29 / 1.91)

S11 的较高 IPC 来自两个阶段的拉动：一是 LUT 构建阶段的跨质心 4 路 SIMD 内积运算，指令吞吐极高；二是精排阶段对 top-500 候选向量进行 NEON FMA 重计算，同样是高密度计算。S8 的内积扫描虽然也使用 NEON SIMD，但其内存带宽压力（每 cycle 需取更多的 float 数据）导致流水线更频繁地等待数据，IPC 受限。

(3) 指令数差异

S11 需要完成三个串联阶段：LUT 构建 ($M \times K$ 次内积)；全库向量的 ADC 扫描；top-rerank_k 候选向量的精排重算。相比之下，S8 只需在 nprobe 个簇内做单一扫描。指令数的差距 (67B / 24B) 直接解释了延迟的差距 (308 μ s vs 164 μ s)，这是算法层面的计算量差异，而非硬件资源利用率差异。

综合来看，S11 的瓶颈在于“**多阶段工作量**”，而非通常认为的访存问题；S8 的瓶颈则是内存带宽的密集程度 (float 数据)。从这里我们也可以看出，为什么两者的线程数扩展瓶颈也具有一定的差异。

(五) 问题探讨

在实验过程中,我们也对一些细节进行了深入探讨。

1. 为什么 Pthread (S9) 在 T=8 时的扩展性弱于 OpenMP (S8)?

观察 IVF 在 $nprobe=16$ 下的线程扩展数据 (表 2): S8 OMP 在 $T=8$ 时延迟从 $T=4$ 的 $136.9 \mu s$ 进一步降至 $108.9 \mu s$, 算法内加速比提升至 $2.87\times$; 而 S9 Pthread 在 $T=8$ 时延迟为 $181.0 \mu s$, 与 $T=4$ 的 $176.6 \mu s$ 几乎相同, 出现明显的扩展瓶颈。这一现象在 PQ-Final 中更为显著: S6 Pthread $T=8$ 的延迟 ($738 \mu s$) 甚至高于 $T=4$ ($668 \mu s$), 出现明显回退。

这一差异主要来源于动态调度的实现层面。OpenMP 的 `schedule(dynamic, 1)` 在 libgomp 实现中通常采用原子指令 (如 `__sync_fetch_and_add`) 或低开销的工作窃取机制来管理任务队列, 锁竞争极少; 而我们手动实现的 Pthread 调度在共享计数器 `next_task_idx` 上使用了 `pthread_mutex_lock/unlock` 配对操作, 每次任务领取都需要一次完整的互斥锁的加锁-解锁周期。

当 $T=8$ 时, 8 个线程频繁争抢同一把互斥锁, 锁的内部状态 (等待队列、futex 系统调用等) 成为热点资源, 争用频率成倍上升, 提升至 $T=16$ 时更是带来了灾难性的后果; 而单簇扫描时间 (对于 $nlist=256$ 、 $nprobe=16$ 的情况, 单簇约 390 个向量, 扫描耗时约为 $5\sim 10 \mu s$) 与锁竞争的开销已经处于同一数量级, 这使得 Pthread 版本的边际收益彻底消失。

这一现象的优化方向之一是把互斥锁替换为原子整数 (如 `std::atomic<int>::fetch_add`), 从而消除内核态切换的开销。验证这一假设需要重构我们的线程调度逻辑, 这里我们不再展开, 但其结论已经很清晰了: 对于细粒度的任务调度, 选择 OpenMP 或借助原子操作往往优于手动构建基于互斥锁的任务队列。这也是为什么, 对于不是深耕并行这一领域的开发者来说, OpenMP 往往优先于 Pthread 成为更加大众的多线程编程的选择。

2. IVF 中 KMeans 聚类使用 L2 距离, 而在线查询使用内积距离, 这一不一致是否会引入偏差?

观察 IVF 的实现 (`ivf.h` 中的 `ivf_kmeans` 函数), 可以看到在聚类阶段使用的是欧氏距离 (L_2):

$$j^*(v_i) = \arg \min_j \|v_i - c_j\|^2$$

而在线查询阶段, 无论是粗排 (计算 query 与簇心距离) 还是精排 (计算 query 与簇内向量距离), 使用的都是内积距离 $d(q, v) = 1 - \langle q, v \rangle$ 。这种“训练距离”与“查询距离”的不一致为什么没有对检索精度造成影响?

实际上, 这种做法在 ANN 领域是一种常见的设计。从理论角度看, 对于单位向量 (即 $\|v\|_2 = 1$), L_2 距离与内积距离存在确定的对应关系:

$$\|x - y\|_2^2 = \|x\|^2 + \|y\|^2 - 2\langle x, y \rangle = 2 - 2\langle x, y \rangle$$

此时 L_2 距离与内积距离的**排序完全等价**, 因此用 L_2 聚类完全不会破坏内积查询的最近邻关系。DEEP100K 数据集的向量已经过归一化处理 (模长接近 1), 这一等价性近似成立。

从工程角度看, 选择 L_2 作为聚类距离主要有两个原因:

- KMeans (Lloyd 算法) 在 L_2 度量下有保证收敛的理论性质: 每次迭代后簇内平方和 (WCSS) 单调递减, 算法必然在有限步内收敛到局部最优。若直接用内积度量做“球面 KMeans” (spherical k-means), 则需要在每次迭代中将簇心重新归一化, 算法复杂度增加且收敛性分析更复杂。(其实实验最开始就是使用最简单的内积距离进行聚类的, 但在这次实验过程中有一回突然出现了收敛问题, 不知道是怎么导致的, 改了很久没能解决, 后来就换用 L_2 距离了。)

- L2 距离的数值稳定性更好。内积量级与向量模长直接相关, 在非归一化场景下可能出现“大模长向量主导聚类”的偏差; 而 L2 距离对模长波动有一定的抵抗能力。

潜在偏差与改进方向: 如果数据没有严格归一化, L2 聚类可能会将”方向相近但模长不同的向量”分入不同簇, 而它们在内积度量下其实是相近的。对于本实验的 DEEP100K 来说, 这一偏差可以忽略, 从实验结果上看 IVF 的召回率 ($nprobe=16$ 时为 0.96) 与理论预期吻合。若在更敏感的应用场景下, 可以考虑; (1) 训练前显式归一化所有向量; (2) 在 KMeans 初始化时使用 KMeans++, 降低对初始随机种子的依赖, 提升聚类质量。

3. IVF-PQ 方法一 (全局 PQ) 与方法二 (簇内独立 PQ) 的特征与差异

从 $nprobe$ 扫描的数据 (表 3 与图 4) 可以看出, 方法一与方法二并非简单的某一个全面占优, 而是在 recall-latency 曲线上具有明显的分段现象。

在中低召回率区间 ($recall \leq 0.93$), **方法一占优**: 例如在召回率约 0.93 处, 方法一在 $nprobe=32$ 下只需 $632 \mu s$, 而方法二在 $nprobe=16$ 下需要 $781 \mu s$ 。其根源在于方法一的 LUT 只需构建一次即可在所有候选簇间复用, 单次查询的固定开销极小。

但在高召回率区间 ($recall \geq 0.94$), 情况则有所不同。方法一无法到达更高召回率, $nprobe$ 增至 64 时召回率仅 0.9359 即出现饱和; 而方法二仍能继续提升, 在 $nprobe=64$ 下达到 0.9668。出现这一现象的根本原因在于, 方法一在所有簇上共享同一套 PQ 质心, 难以精确刻画各簇内部局部结构的差异, 量化误差存在系统性的下界; 而方法二在每个簇内独立训练量化器, 量化精度天然更高。

由此我们可以知道, 面对不同的需求, 可以根据情况选用不同的方法:

- 若目标召回率 ≤ 0.93 , 优先选择**方法一**, 其延迟更低, 且仅需单次 LUT 构建;
- 若目标召回率 ≥ 0.94 , 此时方法一已经触及精度瓶颈, 优先选择**方法二**, 而其代价是延迟有所上升;
- 若同时启用多线程优化 ($T=4$), 则两种方法的延迟差距会进一步减少 ($S11=260 \mu s$ vs $S14=380 \mu s$), 此时如果对召回率有较高要求, 方法二在多线程下也具有更好的性价比。

五、总结与体会

在本次实验中, 我们在 Lab 2 SIMD 优化的基础上, 进一步探索了多线程并行编程在 ANN 问题中的应用, 实现并对比了 OpenMP 与 Pthread 两种并行框架在多种算法中的表现。

从实验结果来看, 多线程优化带来了显著的性能提升: **Flat T=8 OMP 达到 $5.96\times$ 加速, IVF OMP T=8 达到 $52.91\times$ 的全局加速比, IVF-PQ M1 OMP T=8 达到 $25.38\times$** 。与此同时, 在实验中我们也体会到了多线程优化的局限性, 当算法从 compute-bound 转向 memory-bound (如 IVF-PQ), 以及当我们手动实现线程调度在 T 较大时带来的显著锁竞争时, 线程数的增加往往无法带来稳定的等比例的性能提升, 甚至线程过多还会导致负优化的结果。尽管课上老师也讲过相关的内容, 但是切身尝试之后还是会有更多不一样的体验。

本次实验最大的收获在于**线程池**的设计与实现。在未引入线程池之前, Pthread 版本在多种算法上都出现了严重负优化, 误导了对并行框架优劣的判断。引入线程池后, Pthread 的实际性能与 OpenMP 趋于一致, 这让我对于并行优化有了更加深入的理解, 也意识到, 在 OpenMP 看似简单的背后, 其实隐藏着精心设计的线程调度逻辑。

在算法层面, IVF 的思路在现在想来其实是非常契合 ANN 问题的常见思路的, 粗排的思想其实是这些算法中最为核心的体现, 也让我模糊地感受到了一种计算的哲学。此外, 探索 HNSW 的多子图并行也让我感受到了图索引的独特之处, 以及分布式搜索在广度上的优势。

更深层的体会在于,本次实验中多次出现了理论与实践不完全一致的现象,例如原本预期“Pthread 灵活性更高, T 较大时优于 OMP”,实际结果却完全相反;又如 IVF-PQ 方法二理论上召回率更高,延迟虽然会增加但不会很多,可是实际测试后发现延迟增加的幅度还是比较明显的,又需要进行更全面的考虑。这次实验中的诸多现象让我意识到,并行优化还是很需要进行实际的实验测试的,每一步都需要建立在具体场景下的实测数据之上,而后才能得出真实而有效的分析。

总体而言,在本次实验中完整实现了从串行到多线程的并行化过程,包括负载均衡、动态调度、线程同步与线程池等核心问题的探讨与分析,进一步加深了我对并行程序设计的理解,也为今后在实际工程中进行多线程优化提供了宝贵的实践经验。

六、 附录: 代码仓库链接

GitHub: <https://github.com/Iriasoul/Parallel-Programming/tree/main/lab03>

NIJUB